

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure en Informatique
08 Mai 1945- Sidi Bel Abbès



Rapport de Projet

Spécialité : Ingénierie des Systèmes Informatiques (ISI)

Plateforme intelligente d'analyse et de détection d'intrusions (IDS)

Encadré par :
Dr. KHALDI Miloud

Réalisé par :
Benzaoucha Asma
Belhouari Essadjidet
Rahmoun Waffaa
Chaala Ines

Année universitaire : 2025–2026

Table des matières

1	Introduction Générale	6
1.1	introduction	6
1.2	Contexte du projet	6
1.3	Problématique	6
1.4	Objectifs	7
1.5	Motivation	7
1.5.1	Intérêt multidisciplinaire : sécurité, réseau, IA, développement . .	7
2	Les Systèmes de Détection d’Intrusions (IDS)	8
3	Architecture Générale de Notre Système de Détection d’Intrusions	10
3.1	Architecture Logique	10
3.2	Architecture Physique	11
4	Méthodologie généralisée pour les IDS fondés sur le ML/DL	12
5	Présentation du Dataset Utilisé	13
5.1	Définition du dataset NF-UQ-NID	14
6	Pipeline de Traitement de Données	15
6.1	Prétraitement : Balancement des Classes	15
6.1.1	Undersampling : Réduction à 400 000 lignes	15
6.1.2	Oversampling : Rééquilibrage par SMOTE	15
6.2	Prétraitement : Nettoyage et Mise à l’Échelle	16
6.3	Nettoyage des données	16
6.4	Standardisation et formalisation	17
6.5	Sélection de caractéristiques	17
7	Conception et Entraînement des Modèles IA	18
7.1	Algorithmes (Machine Learning)	18
7.1.1	K-Nearest Neighbors (KNN)	18
7.1.2	Support Vector Machine (SVM)	18
7.1.3	Logistic Regression	19
7.1.4	Decision Tree	19
7.1.5	Random Forest	19
7.1.6	Naive Bayes	19
7.1.7	XGBoost	19
7.1.8	Adaboost	19
7.2	Apprentissage Profond (Deep Learning)	20

7.2.1	Réseaux LSTM (Long Short-Term Memory)	20
7.2.2	Perceptron Multicouche (Multilayer Perceptron – MLP)	20
7.2.3	Réseaux de Neurones Profonds (DNN – Deep Neural Networks)	21
7.3	Entraînement des Modeles	21
7.4	Evaluation des performances	22
8	Infrastructure Réseau et Capture de Données Réelles	25
8.1	Définitions de base : Paquet et Flux réseau	25
8.2	Infrastructure	25
8.2.1	Sniffing du Réseau	25
8.2.2	Extraction des Champs	26
8.2.3	Les champs Extraits	26
8.2.4	Formatage du CSV et Prédiction via XGBoost	27
8.3	Configuration réseau : Surveillance via Switch Mirroring	29
8.3.1	Définition	29
8.3.2	Objectifs	29
8.3.3	Modes de duplication disponibles	29
8.3.4	Objectif global de l’architecture	29
8.3.5	Matériel requis	29
8.4	Connexions physiques	29
8.5	Configuration du port mirroring	30
8.6	Mise en place de l’analyse sur le PC IDS	30
8.7	Simulation d’une attaque réseau	30
8.8	Attaque de Reconnaissance (Reconnaissance Attack)	31
8.9	Attaque par Porte Dérobée (Backdoor)	32
9	Développement de la Plateforme	35
9.1	Conception de l’interface (frontend)	35
9.2	Intégration du modèle de détection	35
9.3	Visualisation des prédictions	35
9.4	Fonctionnalité de chatbot pour l’assistance de l’administrateur	35
10	Implémentation Technique	36
10.1	Technologies utilisées	36
10.2	Architecture de la solution	37
10.3	Entraînement et Évaluation du Modèle	37
10.3.1	Entraînement du Modèle	37
10.3.2	Évaluation des Performances du Modèle	38
10.4	Implémentation	38
10.4.1	Détails d’Implémentation	39
11	Défis et Problèmes Rencontrés	45
11.1	Contraintes Matérielles	45
11.2	Problèmes de Données	45
12	Perspectives d’Amélioration	46
12.1	Passage au Temps Réel et Réponse Active	46
12.2	Évolution de la Plateforme	46

Table des figures

2.1	Classification des ID	8
3.1	Architecture logique	10
3.2	Architecture physique	11
4.1	Approche ML/DL généralisée pour IDS	12
6.1	Page d'équilibrage des données	16
10.1	Page d'atterrissage	39
10.2	Page de Sign Up	40
10.3	Page Log In	41
10.4	Page de Home Page	41
10.5	Page d'Archive	42
10.6	Page de scan	43
10.7	Page de chat bot	44

1 Introduction Générale

1.1 introduction

La sécurité des réseaux est aujourd'hui un enjeu majeur face à la montée des cyberattaques. Les systèmes de détection d'intrusions (IDS) traditionnels, basés sur des signatures, montrent leurs limites face aux menaces nouvelles ou modifiées. Pour y remédier, ce projet propose une approche basée sur le Machine Learning.

1.2 Contexte du projet

À l'ère du numérique, la sécurité de l'information est devenue une priorité mondiale. La prolifération des appareils connectés, l'essor du cloud computing, l'Internet des objets (IoT), et la digitalisation croissante des services ont considérablement élargi la surface d'attaque des systèmes informatiques. Ces évolutions ont donné lieu à une explosion des cybermenaces : attaques par déni de service (DDoS), intrusions malveillantes, ransomwares, vols de données personnelles, et autres formes de cybercriminalité.

La cybersécurité a donc pour mission de garantir la confidentialité, l'intégrité et la disponibilité des données, tout en assurant la continuité des services critiques. Elle s'appuie sur un ensemble de technologies, de processus et de bonnes pratiques destinés à protéger les réseaux, les systèmes et les utilisateurs contre les attaques internes ou externes. Dans ce contexte, la capacité à détecter rapidement et efficacement une attaque devient essentielle

1.3 Problématique

Malgré les progrès des systèmes de sécurité, les cyberattaques deviennent de plus en plus sophistiquées et difficiles à détecter. Les systèmes classiques basés sur des signatures montrent leurs limites face à des attaques inconnues ou polymorphes. De plus, un défi majeur pour les systèmes de détection est le **taux élevé de fausses alertes** (FAR/FPR), qui rend difficile la prise de décision et peut saturer les ressources humaines et techniques.

Il est donc nécessaire de concevoir des systèmes capables non seulement de **détecter avec précision** les intrusions, mais aussi de **réduire les faux positifs**, tout en s'adaptant à de nouveaux types de menaces. Cela ouvre la voie à l'intégration de techniques d'intelligence artificielle et d'apprentissage automatique pour améliorer les capacités de détection. :

1.4 Objectifs

L'objectif principal de ce travail est de développer **une plateforme intelligente de détection d'intrusions (IDS)** basée sur des modèles d'intelligence artificielle. Cette plateforme sera capable d'analyser le trafic réseau en temps réel afin de distinguer automatiquement les comportements normaux des comportements malveillants. En s'appuyant sur l'apprentissage automatique et l'apprentissage profond, notre système vise à renforcer la sécurité informatique en détectant efficacement les attaques, y compris celles qui sont nouvelles ou difficiles à identifier par les méthodes traditionnelles

1.5 Motivation

1.5.1 Intérêt multidisciplinaire : sécurité, réseau, IA, développement

Ce projet nous a permis de mobiliser des compétences variées dans plusieurs domaines clés :

- **Sécurité informatique** : Compréhension des types d'attaques réseau, des vecteurs de menace, et des stratégies de détection ;
- **Réseaux informatiques** : Configuration d'environnements de capture, manipulation de paquets, extraction de caractéristiques depuis les flux (features) ;
- **Science des données** : Traitement de grandes quantités de données, nettoyage, équilibrage, sélection de variables ;
- **Intelligence artificielle** : Mise en œuvre et optimisation d'algorithmes de classification ;
- **Développement logiciel** : Création d'une interface utilisateur (frontend), intégration du backend, visualisation en temps réel, et ajout de services comme un chatbot.

2 Les Systèmes de Détection d’Intrusions (IDS)

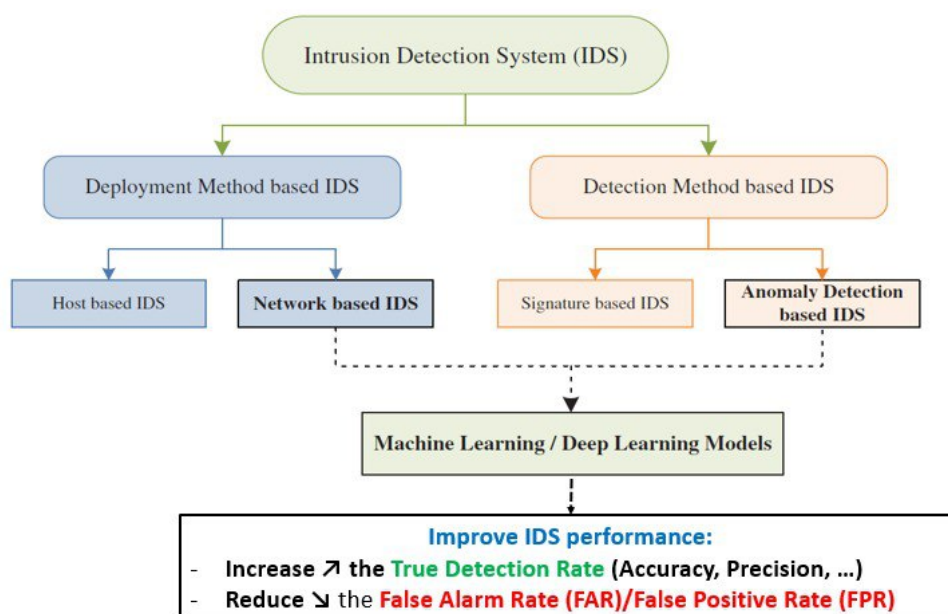


FIGURE 2.1 – Classification des ID

Définition

Un **IDS (Intrusion Detection System)** est un système conçu pour surveiller l'activité réseau ou système, afin de détecter toute activité suspecte ou malveillante. Il agit comme une alarme de sécurité numérique, signalant toute tentative de compromission d'un réseau ou d'un hôte.

Classification des IDS

Les IDS peuvent être classés selon deux critères principaux :

A. Selon la Méthode de Déploiement

- **Host-Based IDS (HIDS)** : déployé directement sur les machines à surveiller, il analyse les fichiers journaux, les appels système et le comportement local.
- **Network-Based IDS (NIDS)** : installé sur un segment du réseau, il examine le trafic circulant entre les machines pour identifier des anomalies ou des attaques.

L'intégration de l'Intelligence Artificielle*

L'usage de modèles d'apprentissage automatique et profond (ML/DL) permet d'automatiser l'analyse du trafic, d'augmenter la capacité de détection et de s'adapter à l'évolution des attaques. Ces modèles peuvent apprendre à reconnaître des schémas complexes, contribuant à améliorer :

- **Host-Based IDS (HIDS)** : La précision de détection (Accuracy, Precision, Recall, F1-Score...)
- **Network-Based IDS (NIDS)** : La réduction du taux de fausses alertes (FAR, FPR)

3 Architecture Générale de Notre Système de Détection d’Intrusions

3.1 Architecture Logique

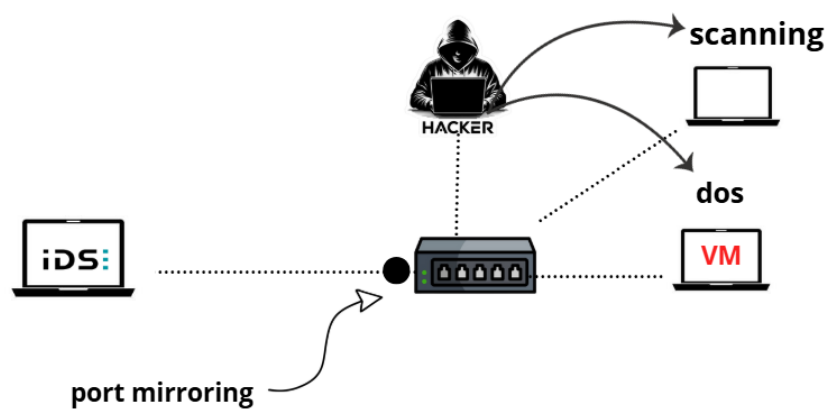


FIGURE 3.1 – Architecture logique

3.2 Architecture Physique



FIGURE 3.2 – Architecture physique

4 Méthodologie généralisée pour les IDS fondés sur le ML/DL

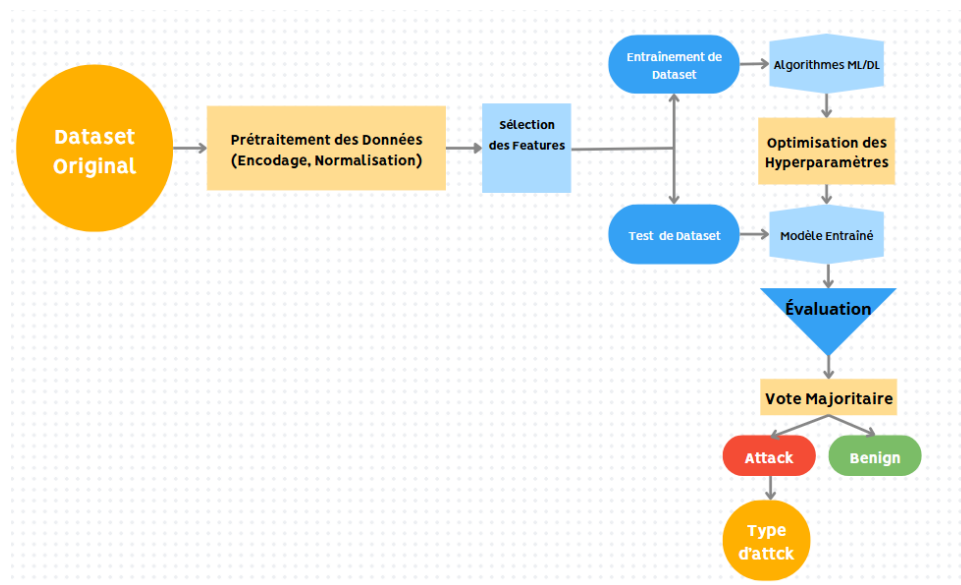


FIGURE 4.1 – Approche ML/DL généralisée pour IDS

5 Présentation du Dataset Utilisé

Caractéristique	Valeur
Nom du dataset	NF-UQ-NIDS-v2
Propriétaire	University of Queensland (UQ)
Année	2021
Nombre d’instances	Plus de 75 millions
Nombre de caractéristiques	48
Nombre de classes	20
Format	NetFlow optimisé
Type de données couvertes	Réseau classique, Cloud, IoT, IIoT
Sous-datasets intégrés	NF-UNSW-NB15-v2, NF-CSE-CIC-IDS2018-v2, NF-BoT-IoT-v2, NF-ToN-IoT-v2
Taille brute	+13 Go
Lien (URL)	NF-UQ-NIDS-v2

TABLE 5.1 – Caractéristiques du dataset NF-UQ-NIDS-v2

Caractéristique	NF-UNSW-NB15-v2	NF-CSE-CIC-IDS2018-v2	NF-BoT-IoT-v2	NF-ToN-IoT-v2
Propriétaire	UNSW Canberra Cyber	Canadian Institute for Cybersecurity (CIC)	UNSW Canberra Cyber	CSIRO Data61 & UNSW Canberra Cyber
Année	2021	2018	2019	2020
Lien	Inclus dans NF-UQ-NIDS-v2	Inclus dans NF-UQ-NIDS-v2	Inclus dans NF-UQ-NIDS-v2	Inclus dans NF-UQ-NIDS-v2

TABLE 5.2 – Comparaison des sous-ensembles de NF-UQ-NIDS-v2

5.1 Définition du dataset NF-UQ-NIDS

Le dataset **NF-UQ-NIDS** est un ensemble de données complet résultant de la fusion de plusieurs datasets majeurs : *NF-UNSW-NB15-v2*, *NF-CSE-CIC-IDS2018-v2*, *NF-BoT-IoT-v2* et *NF-ToN-IoT-v2*. Cette version consolidée met en évidence les avantages de l'intégration de jeux de données partageant des caractéristiques communes, rendant possible la création d'un grand dataset universel pour les systèmes de détection d'intrusion (**NIDS**).

Il regroupe des flux issus de divers environnements réseau (réseaux classiques, **IoT**, **cloud**, **entreprise**) et couvre une large variété de types d'attaques.

Le dataset final **NF-UQ-NIDS** contient un total de **75 987 976** flux, dont **25 165 295** (soit **33,12 %**) sont des flux bénins, et **50 822 681** (soit **66,88 %**) représentent des attaques.

Classe	Nombre d'occurrences
Benign	25 165 295
DDoS	21 748 351
Reconnaissance	2 633 778
Injection	684 897
DoS	17 875 585
Brute Force	123 982
Password	1 153 323
XSS	2 455 020
Infiltration	116 361
Exploits	31 551
Scanning	3 781 419
Fuzzers	22 310
Backdoor	18 978
Bot	143 097
Generic	16 560
Analysis	2 299
Theft	2 431
Shellcode	1 427
MITM	7 723
Worms	164
Ransomware	3 425

TABLE 5.3 – Répartition des classes dans le dataset NF-UQ-NIDS

6 Pipeline de Traitement de Donnees

6.1 Prétraitement : Balancement des Classes

6.1.1 Undersampling : Réduction à 400 000 lignes

- Tri par type d'attaque.
- Lecture par blocs pour éviter la surcharge mémoire.
- Sélection de 400 000 lignes par type d'attaque.

Méthode choisie : *KNN-Order Undersampling* :

- Préserve la structure locale des données.
- Meilleurs résultats que le tirage aléatoire.

Sources : <https://www.scirp.org/journal/paperinformation?paperid=60996>, <https://www.sciencedirect.com/science/article/pii/S1877050919313456>

6.1.2 Oversampling : Rééquilibrage par SMOTE

Pour les classes à moins de 400 000 lignes, la méthode SMOTE (Synthetic Minority Oversampling Technique) a été appliquée.

Dataset final : 400 000 lignes par classe, incluant : 2

- Analysis
- Backdoor
- Benign
- Bot
- Brute Force
- DDoS
- DoS
- Exploits
- Fuzzers
- Generic
- Infiltration
- Reconnaissance
- Shellcode
- Theft
- Injection

- MITM
- Password
- Scanning
- XSS
- Ransomware
- Worms

Page d'équilibrage des données

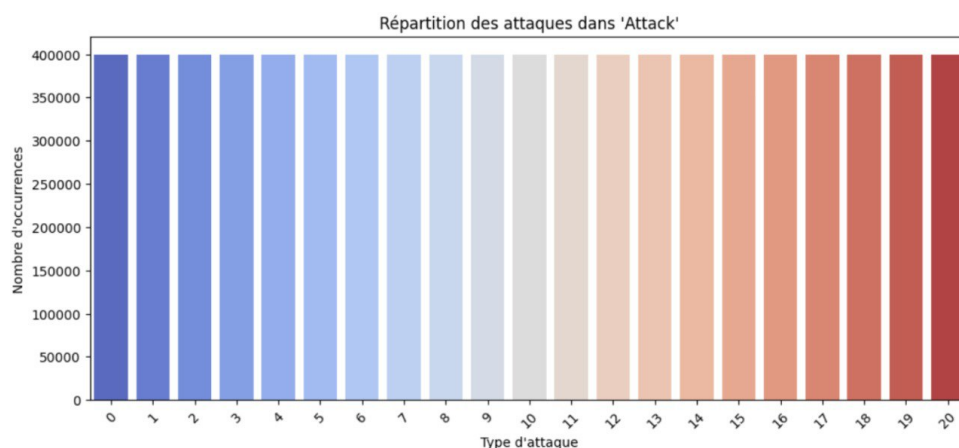


FIGURE 6.1 – Page d'équilibrage des données

6.2 Prétraitement : Nettoyage et Mise à l'Échelle

Le pré-traitement constitue une étape fondamentale dans tout projet d'analyse ou de modélisation en intelligence artificielle. Une donnée mal préparée entraîne inévitablement des performances faibles, voire erronées, même avec les meilleurs modèles. Dans notre cas, cette phase a été cruciale pour garantir un apprentissage efficace.

6.3 Nettoyage des données

Le dataset brut contenait plusieurs valeurs manquantes, doublons et erreurs de type. Nous avons appliqué une série d'opérations de nettoyage, notamment :

- Suppression des lignes incomplètes (valeurs NaN ou nulles dans les colonnes critiques).
- Élimination des doublons pouvant fausser les statistiques.
- Conversion des types de données incorrects (ex. : certaines colonnes numériques encodées en chaîne de caractères).
- Suppression des caractères spéciaux ou inutiles.

6.4 Standardisation et formalisation

Afin que les algorithmes d'apprentissage puissent interpréter les données correctement, nous avons procédé à :

- La standardisation des données numériques : transformation des valeurs pour qu'elles aient une moyenne de 0 et un écart-type de 1 via MinMaxEncoder.
- Encodage des variables catégorielles : certaines colonnes comme les types de protocoles (TCP, UDP, etc.) ont été transformées en variables numériques via Label Encoding.
- Transformation des étiquettes de classes : les types d'attaques (DoS, infiltration, etc.) ont été encodés pour être compréhensibles par les modèles.

6.5 Sélection de caractéristiques

Étant donné le grand nombre de caractéristiques disponibles dans notre jeu de données, il était essentiel de réduire la dimensionnalité tout en conservant les informations pertinentes. Pour cela, nous avons utilisé plusieurs méthodes :

- **Random Forest (RF)** : nous avons extrait les scores d'importance des caractéristiques à partir de ce modèle pour identifier les plus influentes.
- **Firefly Algorithm** : une méthode d'optimisation inspirée du comportement des lucioles, utilisée pour rechercher le sous-ensemble optimal de caractéristiques.
- **PCA (Principal Component Analysis)** : utilisée en complément pour réduire la dimensionnalité en transformant les données en un nouvel espace de variables non corrélées.

L'objectif était de réduire la dimensionnalité du jeu de données sans perte d'information critique, ce qui améliore à la fois les performances et la vitesse d'entraînement.

7 Conception et Entraînement des Modèles IA

La détection d'intrusions dans les systèmes informatiques nécessite des modèles performants, capables de distinguer le trafic normal du trafic malveillant. Dans notre projet, nous avons exploré à la fois des algorithmes de Machine Learning classiques et des approches plus avancées d'Apprentissage Profond.

L'objectif était de comparer les performances des modèles et de sélectionner les plus efficaces en termes de précision, rappel, F1-score et robustesse face à la diversité des attaques.

7.1 Algorithmes (Machine Learning)

Cette section présente les principaux algorithmes d'apprentissage automatique que nous avons explorés ou utilisés dans le cadre de ce projet. Chacun de ces algorithmes offre des caractéristiques distinctes qui les rendent adaptés à différentes facettes de la détection d'intrusion.

7.1.1 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) est un algorithme d'apprentissage non paramétrique, basé sur les instances, utilisé à la fois pour la classification et la régression. Pour une instance de test donnée, il identifie les k points de données les plus proches dans l'ensemble d'entraînement en utilisant une métrique de distance (par exemple, la distance euclidienne) et assigne l'étiquette de classe basée sur le vote majoritaire parmi ces voisins. L'algorithme est sensible au choix de k et à la métrique de distance utilisée.

7.1.2 Support Vector Machine (SVM)

Support Vector Machine (SVM) est un puissant algorithme d'apprentissage supervisé utilisé pour les tâches de classification et de régression. Il construit un hyperplan ou un ensemble d'hyperplans dans un espace de grande dimension pour séparer les classes avec la plus grande marge possible. Les SVM peuvent efficacement effectuer une classification non linéaire en utilisant l'astuce du noyau (kernel trick), qui consiste à projeter les entrées dans des espaces de caractéristiques de dimension supérieure.

7.1.3 Logistic Regression

La Régression Logistique est un modèle statistique principalement utilisé pour la classification binaire. Il prédit la probabilité d'un résultat binaire basé sur une ou plusieurs variables prédictives en utilisant une fonction logistique (ou sigmoïde). Cet algorithme est particulièrement efficace pour les problèmes où les classes sont linéairement séparables ou quasi séparables.

7.1.4 Decision Tree

Un Arbre de Décision est un algorithme d'apprentissage supervisé non paramétrique utilisé pour la classification et la régression. Il construit un modèle de décisions en forme d'arbre basé sur les caractéristiques des données, divisant les données en sous-ensembles en fonction de la valeur des caractéristiques jusqu'à ce qu'une décision ou une prédiction soit faite. Il est interprétable et peut gérer à la fois des données numériques et catégorielles.

7.1.5 Random Forest

Random Forest est une méthode d'apprentissage d'ensemble (ensemble learning) qui utilise plusieurs arbres de décision pour améliorer la précision de la classification ou de la régression. Il construit de nombreux arbres pendant la phase d'entraînement et produit le mode des classes (pour la classification) ou la moyenne des prédictions (pour la régression) des arbres individuels. Cette approche réduit le surapprentissage et améliore la robustesse.

7.1.6 Naive Bayes

Naïve Bayes est un classificateur probabiliste basé sur le théorème de Bayes, avec une hypothèse forte d'indépendance entre chaque paire de caractéristiques (d'où le terme "naïf"). Il est simple à implémenter, rapide et souvent très efficace pour de nombreux types de tâches de classification, notamment avec des données de grande dimension ou des données textuelles.

7.1.7 XGBoost

XGBoost (Extreme Gradient Boosting) est une implémentation avancée de la méthode d'apprentissage par boosting de gradient pour les problèmes d'apprentissage supervisé. Il utilise une série d'arbres de décision où chaque nouvel arbre corrige les erreurs des arbres précédents. XGBoost est réputé pour son optimisation des performances, sa vitesse d'exécution et sa capacité à gérer efficacement des ensembles de données à grande échelle. Il intègre également des techniques de régularisation pour éviter le surapprentissage.

7.1.8 Adaboost

AdaBoost est une méthode d'ensemble qui fonctionne en entraînant plusieurs modèles faibles de manière séquentielle, où chaque nouveau modèle se concentre davantage sur les exemples mal classés par les modèles précédents. À la fin, il combine tous les modèles pour faire une prédiction finale pondérée.

7.2 Apprentissage Profond (Deep Learning)

L'apprentissage profond (Deep Learning – DL) est un domaine spécialisé du machine learning qui repose sur l'utilisation de réseaux de neurones comportant plusieurs couches (d'où le terme "profond"). Ces architectures neuronales complexes sont capables de modéliser des relations très complexes dans de vastes ensembles de données, allant bien au-delà des capacités des modèles traditionnels.

Les algorithmes de deep learning ont la particularité d'apprendre automatiquement des représentations pertinentes et hiérarchiques des données brutes, éliminant ainsi le besoin d'une ingénierie manuelle des caractéristiques. Cette capacité les rend particulièrement efficaces pour des tâches impliquant des entrées à haute dimension et non structurées, comme les images, les sons, les vidéos ou les textes. Leur succès fulgurant s'explique notamment par l'augmentation exponentielle de la puissance de calcul (notamment grâce aux GPU), la disponibilité de grands volumes de données pour l'entraînement, et l'évolution constante des techniques d'entraînement et des architectures de réseaux neuronaux profonds.

7.2.1 Réseaux LSTM (Long Short-Term Memory)

Les réseaux LSTM (Long Short-Term Memory) sont une variante spécialisée des réseaux neuronaux récurrents (RNN), spécifiquement conçue pour gérer et apprendre des dépendances à long terme dans les données séquentielles. Contrairement aux RNN classiques qui peinent à retenir des informations sur de longues séquences, les LSTM intègrent des "cellules de mémoire" et des mécanismes de "portes" (porte d'entrée, porte d'oubli, porte de sortie) qui régulent de manière sélective la circulation de l'information. Cette architecture sophistiquée leur permet de stocker ou d'ignorer des informations sur de très longues périodes. Cela les rend particulièrement adaptés à des tâches où l'ordre et le contexte sont cruciaux, telles que la prédiction de séries temporelles, la reconnaissance vocale, la traduction automatique ou la modélisation du langage naturel.

7.2.2 Perceptron Multicouche (Multilayer Perceptron – MLP)

Le Perceptron Multicouche (Multilayer Perceptron – MLP) est un type fondamental de réseau de neurones artificiels "feedforward" (à propagation avant). Il est composé d'au moins trois couches de nœuds : une couche d'entrée, une ou plusieurs couches cachées (appelées couches intermédiaires), et une couche de sortie. Chaque neurone (ou perceptron) dans les couches cachées et de sortie utilise une fonction d'activation non linéaire (comme la fonction sigmoïde, ReLU ou tanh), ce qui permet au réseau de modéliser des relations non linéaires complexes entre les entrées et les sorties. L'apprentissage du réseau repose sur l'algorithme de rétropropagation du gradient (backpropagation), qui vise à minimiser une fonction de perte (erreur) en ajustant itérativement les poids et les biais des connexions entre les neurones. Le MLP est un modèle polyvalent utilisé pour la classification, la régression et la reconnaissance de formes.

7.2.3 Réseaux de Neurones Profonds (DNN – Deep Neural Networks)

Les réseaux de neurones profonds (DNN) sont une catégorie d'architectures d'apprentissage profond composées de plusieurs couches de neurones artificiels. Chaque couche est responsable de l'extraction progressive de caractéristiques de plus haut niveau à partir des données d'entrée.

Un DNN est structuré comme suit :

Couche d'entrée : reçoit les données brutes (par exemple, les caractéristiques extraites du trafic réseau).

Couches cachées multiples : ces couches intermédiaires effectuent des transformations non linéaires successives, permettant d'apprendre des représentations complexes et abstraites des données.

Couche de sortie : génère la prédiction finale, par exemple, la classification d'un paquet réseau comme « normal » ou « malveillant ».

7.3 Entraînement des Modeles

Pour entraîner les modèles de détection, nous avons suivi plusieurs étapes clés :

— **Séparation du dataset :**

- 80% pour l'entraînement,
- 20% pour le test.

— **Optimisation des hyperparamètres :**

- **GridSearchCV** : méthode d'exploration exhaustive sur un ensemble défini de combinaisons d'hyperparamètres. Elle entraîne et valide le modèle pour chaque configuration possible afin de trouver la meilleure.

ML algorithms	Parameters	Optimal value
LR	Penalty : C	10
NB	Var smoothing	$1e^{-10}$
KNN	K	3
DT	Criterion	gini
	min_samples_leaf	5
	min_samples_split	9
RF	n_estimators	100
	min_samples_leaf	1
	min_samples_split	5
AdaBoost	n_estimators	1000
	learning_rate	1
SVM	Kernel	rbf
	C	100,000
	γ	10,000

TABLE 7.1 – Paramètres et valeurs optimales des algorithmes de ML

- **PSO (Particle Swarm Optimization)** : algorithme d'optimisation inspiré du comportement collectif des oiseaux ou poissons. Chaque "particule" représente une solution candidate, qui se déplace dans l'espace de recherche en fonction de sa propre expérience et celle des autres.

7.4 Evaluation des performances

Les modèles ont été évalués avec les métriques suivantes :

- **Accuracy** : taux de prédictions correctes.
- **Précision** : proportion de vrais positifs parmi les positifs prédits.
- **Recall** : proportion de vrais positifs détectés parmi les cas réels.
- **F1-score** : moyenne harmonique entre précision et rappel.
- **FAR / FPR** : taux de fausses alertes.
- **ROC-AUC** : surface sous la courbe ROC.
- **MCC** : coefficient de corrélation de Matthews.
- **Matrice de confusion** : visualisation des erreurs de classification.

Avant Équilibrage

TABLE 7.2 – Résultats des modèles avant balancement

Métrique	XGBoost	AdaBoost	Logistic Regression	MLP	DNN
Accuracy	0.9402	0.5160	0.7543	0.8988	0.9148
Precision	0.9395	0.4643	0.7475	0.8957	0.9140
Recall	0.9402	0.5160	0.7543	0.8988	0.9148
F1-Score	0.9366	0.4476	0.7417	0.8895	0.9056
FAR / FPR	0.0032	0.0260	0.0132	0.0054	0.0046
ROC-AUC Score	0.9978	0.8940	0.9640	0.9935	0.9954
Matthews Corr. Coeff (MCC)	0.9331	0.4769	0.7260	0.8868	0.9048

TABLE 7.3 – Résultats des modèles après balancement

Mesures	XGBoost	AdaBoost	Logistic Regression	MLP	DNN	LSTM
Accuracy	0.9790	0.4777	0.8752	0.9608	0.9573	0.9650
Precision	0.9833	0.5482	0.8818	0.9655	0.9622	0.9697
Recall	0.9790	0.4777	0.8752	0.9608	0.9573	0.9650
F1-Score	0.9798	0.4249	0.8747	0.9613	0.9581	0.9657
FAR / FPR	0.0010	0.0261	0.0062	0.0020	0.0021	0.0017
ROC-AUC Score	0.9992	0.9631	0.9935	0.9985	0.9985	0.9988
MCC	0.9782	0.4711	0.8694	0.9591	0.9554	0.9635

TABLE 7.4 – Résultats après PCA (réduction de 44 → 27)

Mesures	XGBoost	AdaBoost	Logistic Regression	MLP	DNN	LSTM
Accuracy	0.9799	0.5859	0.8846	0.9607	0.9670	0.9678
Precision	0.9840	0.5864	0.8929	0.9647	0.9715	0.9723
Recall	0.9799	0.5859	0.8846	0.9607	0.9670	0.9678
F1-Score	0.9807	0.5496	0.8848	0.9611	0.9676	0.9684
FAR / FPR	0.0010	0.0207	0.0058	0.0020	0.0017	0.0015
ROC-AUC Score	0.9992	0.9619	0.9939	0.9984	0.9987	0.9988
MCC	0.9791	0.5695	0.8793	0.9589	0.9656	0.9664

Après l'importance des caractéristiques

Mesures	XGBoost	AdaBoost
Accuracy	0.9818	0.4902
Précision	0.9856	0.3974
Recall	0.9818	0.4902
F1-Score	0.9825	0.41126
FAR / FPR	0.0009	0.0255
ROC-AUC Score	0.9993	0.9406
MCC	0.9810	0.4966

Après PSO

Meilleurs paramètres :

C = 9.62554, max_iter = 855

Mesures	Régression Logistique
Accuracy	0.8177
Précision	0.8219
Recall	0.8177
F1-Score	0.8138
FAR / FPR	0.0091
ROC-AUC Score	0.9875
MCC	0.8093

TABLE 7.5 – Performance de l'algorithme Decision Tree avant sélection de caractéristiques

Modèle	Accuracy	Précision	Recall	F1-Score	FAR/FPR	ROC-AUC	MCC
Decision Tree	0.9813	0.9847	0.9813	0.9820	0.0000	0.9975	0.9805

TABLE 7.6 – Performance de l’algorithme Decision Tree après réglage des hyperparamètres

Modèle	Accuracy	Précision	Recall	F1-Score	FAR/FPR	ROC-AUC	MCC
Decision Tree	0.9824	0.9862	0.9824	0.9831	0.0000	0.9991	0.9817

TABLE 7.7 – Performance de Random Forest avant et après la sélection de caractéristiques

Étape	Accuracy	Précision	Recall	F1-Score	FAR/FPR	ROC-AUC	MCC
Avant sélection	0.9644	0.9646	0.9645	0.9644	0.0000	0.9961	0.9652
Après sélection	0.9820	0.9855	0.9820	0.9827	0.0000	0.9989	0.9813

TABLE 7.8 – Évolution du F1-Score selon le nombre de caractéristiques retenues

Nombre de Features	F1-Score
39	0.9819
40	0.9827
41	0.9827
42	0.9827
43	0.9827
44	0.9827

F1-Score : Combine précision et rappel de façon équilibrée, ce qui en fait une métrique idéale lorsque les faux positifs et les faux négatifs ont un impact similaire. Dans notre cas, le meilleur score F1 atteint est de 0,9827, obtenu de manière stable avec 40 caractéristiques. L’utilisation de toutes les 44 caractéristiques n’a pas permis d’améliorer davantage ce score, ce qui suggère que 4 caractéristiques peuvent potentiellement être éliminées sans perte de performance, contribuant ainsi à une réduction de la complexité du modèle.

TABLE 7.9 – Performance de Naïve Bayes avant et après PCA

Étape	Accuracy	Précision	Recall	F1-Score	FAR/FPR	ROC-AUC	MCC
Avant PCA	0.6639	0.7442	0.6639	0.6364	0.0000	0.9704	0.6539
Après PCA	0.7329	0.7545	0.7329	0.7319	0.0000	0.9698	0.7207

8 Infrastructure Réseau et Capture de Données Réelles

8.1 Définitions de base : Paquet et Flux réseau

Avant de présenter l'infrastructure de notre système de détection d'intrusion, il est essentiel de comprendre deux notions fondamentales du trafic réseau : le paquet et le flux réseau.

Un paquet est la plus petite unité de données transmise sur un réseau informatique. Il contient à la fois les données utiles (payload) et des informations de contrôle (en-têtes) telles que les adresses source et destination, les ports utilisés et le protocole de communication (TCP, UDP, ICMP, etc.). C'est sur la base de ces paquets que tout échange d'information sur Internet s'effectue.

Un flux réseau (ou *network flow*) désigne une séquence de paquets échangés entre deux entités (par exemple, un client et un serveur) qui partagent les mêmes caractéristiques :

- Adresse IP source (IPV4_SRC_ADDR)
- Adresse IP destination (IPV4_DST_ADDR)
- Port source (L4_SRC_PORT)
- Port destination (L4_DST_PORT)
- Protocole de transport (PROTOCOL, tel que TCP, UDP, ICMP, etc.)

Un flux peut être unidirectionnel ou bidirectionnel, selon qu'il s'agisse d'une simple requête ou d'une session interactive. L'analyse des flux permet de détecter des comportements suspects ou malveillants, tels que les attaques par déni de service distribué (DDoS), les balayages de ports ou encore les tentatives d'infiltration.

8.2 Infrastructure

8.2.1 Sniffing du Réseau

Dans le cadre de la capture réseau, l'interface réseau spécifique connectée à l'infrastructure (Ethernet, Wi-Fi, etc.) est sélectionnée pour intercepter le trafic. Cette interface joue un rôle stratégique en tant que point de surveillance, permettant la collecte de tous les paquets entrants et sortants. Le choix de cette interface est crucial : elle doit correspondre au segment réseau analysé, tel qu'une interface configurée en mode « port mirroring » sur un switch afin de capturer l'ensemble du trafic d'un VLAN donné.

Une fois l'interface sélectionnée, le système initie une écoute passive, enregistrant chaque échange de données dans un format standard, typiquement PCAP. Ces données

brutes constituent la source principale d'informations pour notre système de détection d'intrusion (IDS), permettant l'identification d'anomalies et d'attaques potentielles.

8.2.2 Extraction des Champs

Outil utilisé : Scapy (bibliothèque Python)

L'analyse des fichiers PCAP est réalisée à l'aide de scripts Python exploitant la bibliothèque Scapy. L'objectif principal est d'extraire les paquets réseau, puis de les regrouper en flux (flows). Un flux est défini comme un ensemble de paquets partageant les mêmes caractéristiques : adresse IP source et destination, port source et destination, ainsi que le protocole de transport (TCP, UDP, ICMP, etc.).

Chaque flux est ensuite enrichi par un ensemble de métriques statistiques permettant une analyse agrégée du trafic. Ces informations facilitent la surveillance, la détection d'anomalies et l'identification de comportements suspects.

8.2.3 Les champs Extraits

a. Informations de base

- IP source / IP destination : `IPV4_SRC_ADDR`, `IPV4_DST_ADDR`
- Port source / Port destination : `L4_SRC_PORT`, `L4_DST_PORT`
- Protocole de transport : `PROTOCOL` (ex. : 6 pour TCP, 17 pour UDP)
- Protocole applicatif détecté : `L7_PROTO` (ex. : HTTP, DNS, FTP)

b. Volume de données

- Octets transférés : `IN_BYTES`, `OUT_BYTES`
- Nombre de paquets : `IN_PKTS`, `OUT_PKTS`
- Débit moyen : `SRC_TO_DST_SECOND_BYTES`, `DST_TO_SRC_SECOND_BYTES`

c. Temps et durée

- Horodatages : `FIRST_TIMESTAMP`, `LAST_TIMESTAMP`
- Durée totale du flux : `FLOW_DURATION_MILLISECONDS`
- Durée moyenne entre paquets : `DURATION_IN`, `DURATION_OUT`

d. Qualité de transmission

- Retransmissions : `RETRANSMITTED_IN_PKTS`, `RETRANSMITTED_OUT_PKTS`
- Volume retransmis : `RETRANSMITTED_IN_BYTES`, `RETRANSMITTED_OUT_BYTES`
- Débit moyen calculé : `SRC_TO_DST_AVG_THROUGHPUT`, `DST_TO_SRC_AVG_THROUGHPUT`

e. Indicateurs TCP

- Drapeaux TCP : `TCP_FLAGS`, `CLIENT_TCP_FLAGS`, `SERVER_TCP_FLAGS`
- Fenêtre maximale TCP : `TCP_WIN_MAX_IN`, `TCP_WIN_MAX_OUT`

f. Taille des paquets

- Plus petit / plus grand paquet : `SHORTEST_FLOW_PKT`, `LONGEST_FLOW_PKT`
- Taille min/max des paquets IP : `MIN_IP_PKT_LEN`, `MAX_IP_PKT_LEN`
- Répartition par taille : `NUM_PKTS_UP_TO_128_BYTES` à `NUM_PKTS_1024_TO_1514_BYTES`

g. Données spécifiques aux protocoles

- TTL : `MIN_TTL`, `MAX_TTL`

ICMP

- Type de message : `ICMP_TYPE`, `ICMP_IPV4_TYPE`

DNS

- Identifiant de requête : `DNS_QUERY_ID`
- Type de requête : `DNS_QUERY_TYPE`
- TTL de réponse : `DNS_TTL_ANSWER`

FTP

- Code de retour : `FTP_COMMAND_RET_CODE` (ex. : 226 pour « transfert terminé », 530 pour « échec d'authentification »)

Ce processus permet de transformer une capture réseau brute en une vue structurée du trafic, essentielle pour les étapes d'analyse et de détection dans une infrastructure de sécurité réseau.

8.2.4 Formatage du CSV et Prédiction via XGBoost

a. Chargement des objets entraînés

Le processus commence par le chargement des objets sauvegardés lors de l'entraînement du modèle :

- **Le scaler** : utilisé pour normaliser les données d'entrée. Il permet de garantir que les nouvelles données sont sur la même échelle que celles utilisées pendant l'entraînement, évitant ainsi les biais.
- **Le modèle XGBoost** : un modèle de classification supervisée, spécialement entraîné pour détecter et catégoriser les types de trafic réseau (normal ou malveillant).

b. Préparation des données à prédire

Les données issues d'un fichier PCAP (ou d'un flux réseau en temps réel) sont d'abord converties en un format tabulaire numérique. Les étapes de nettoyage incluent :

- Conversion des chaînes non numériques, erreurs ou valeurs manquantes en `NaN`.
- Remplacement des `NaN` par 0 pour assurer la compatibilité avec le modèle.
- Sélection exclusive des colonnes utilisées à l'entraînement du modèle, assurant la cohérence avec les attentes du classifieur.

c. Normalisation des données

Les données filtrées sont ensuite normalisées à l'aide du scaler précédemment chargé.

Cette normalisation aligne toutes les valeurs sur une échelle commune. Elle permet d'éviter qu'une caractéristique ayant des valeurs très élevées (comme la taille d'un paquet) ne domine les autres lors de la classification.

d. Application de la prédiction XGBoost

Les données normalisées sont passées au modèle XGBoost, qui attribue à chaque ligne une étiquette prédite :

- 2 → trafic normal
- 11 → attaque de type reconnaissance
- 6 → attaque de type DoS (Déni de Service)
- (et d'autres classes selon l'entraînement du modèle)

e. Analyse des résultats

Une fois les prédictions obtenues, plusieurs statistiques sont générées :

- Nombre total de lignes classées comme attaque DoS (étiquette = 6)
- Nombre de lignes classées comme trafic normal ou autres attaques

Ces statistiques permettent d'évaluer rapidement la nature et la gravité du trafic observé.

f. Enregistrement dans un fichier CSV

Le résultat final (contenant les caractéristiques normalisées + la prédiction **ATTACK**) est sauvegardé dans un fichier CSV :

- Le fichier est stocké dans un dossier spécifique du projet.
- Son nom est dérivé de celui du fichier analysé (ex. : `capture1_prediction.csv`).

Cela facilite la traçabilité, l'exploitation des résultats, ainsi que leur visualisation graphique.

g. Détermination du type global de trafic

Une analyse globale est ensuite réalisée sur l'ensemble des prédictions :

- On identifie la classe la plus fréquente dans les résultats (mode).
- Cette classe dominante représente le type principal de trafic détecté dans le fichier analysé.

Exemple Si la majorité des lignes sont étiquetées comme 2, le fichier est globalement classé comme une attaque DoS.

8.3 Configuration réseau : Surveillance via Switch Mirroring

8.3.1 Définition

Le mirroring de port (ou *port mirroring*) est une fonctionnalité offerte par les switchs manageables permettant de dupliquer le trafic réseau d'un ou plusieurs ports (entrant, sortant ou les deux) vers un port de destination spécifique. Ce port est généralement relié à un dispositif de surveillance tel que Wireshark, un IDS (*Intrusion Detection System*), ou tout autre outil d'analyse réseau.

8.3.2 Objectifs

- Observer le trafic réseau sans perturber les communications existantes.
- Capturer et analyser les paquets à des fins de diagnostic ou d'étude de sécurité.
- Détecter des comportements anormaux ou des attaques (ex. reconnaissance, DoS).

8.3.3 Modes de duplication disponibles

- **Ingress** : copie uniquement le trafic entrant vers le port ciblé.
- **Egress** : copie uniquement le trafic sortant.
- **Both** : copie l'ensemble du trafic (entrant et sortant).

8.3.4 Objectif global de l'architecture

Mettre en place une infrastructure permettant à une machine IDS de surveiller en continu le trafic échangé entre une machine attaquante et une machine cible, tout en préservant l'intégrité et la fluidité de la communication réelle.

8.3.5 Matériel requis

- Un switch manageable : compatible avec la fonctionnalité de port mirroring (SPAN/RSPAN).
- Un PC attaquant : simulant les attaques (ex. distribution Kali Linux).
- Un PC victime : cible potentielle du trafic malveillant (ex. système Windows ou Linux).
- Un PC IDS : système chargé de l'analyse (ex. Snort, Suricata ou solution maison basée sur Scapy/NFStreamer).
- Câbles RJ45 : pour relier l'ensemble des équipements au switch.

8.4 Connexions physiques

L'architecture repose sur la connexion des équipements suivants à un switch manageable :

- **Port 1** : connecté au PC attaquant (simulateur d'attaque – ex. Kali Linux)
- **Port 2** : connecté au PC victime (machine cible)

- **Port 3** : connecté au PC IDS (système de détection d'intrusion)

Cette disposition permet de surveiller le trafic échangé entre l'attaquant et la victime via le port miroir dédié à l'analyse.

8.5 Configuration du port mirroring

La configuration dépend du modèle du switch (Cisco, HP, Netgear, etc.) et peut être effectuée via une interface web ou en ligne de commande (CLI). L'exemple ci-dessous illustre une configuration en CLI pour un switch Cisco.

Objectif de configuration

- **Port source (ou surveillé)** : port 2 (PC victime), sur lequel le trafic doit être surveillé.
- **Port de destination (ou miroir)** : port 3 (PC IDS), qui recevra une copie du trafic.

L'objectif est de rediriger tout le trafic entrant vers la victime (c'est-à-dire reçu sur le port 2) vers le port 3, sans interrompre la communication réelle.

Exemple de commande Cisco (CLI)

- `monitor session 1 source interface fastEthernet0/2 rx`
- `monitor session 1 destination interface fastEthernet0/3`
- `rx` (receive only) : indique que seul le trafic entrant sur le port 2 est copié.
- L'option `both` peut être utilisée si l'on souhaite surveiller à la fois le trafic entrant et sortant.

8.6 Mise en place de l'analyse sur le PC IDS

Une fois la redirection de trafic en place, un outil d'analyse doit être lancé sur le PC IDS. Plusieurs choix sont possibles :

- Wireshark ou tcpdump pour une capture simple de paquets
- Snort, Suricata ou un outil personnalisé basé sur le machine learning (ex. avec Scapy, NFStreamer) pour une détection intelligente

Le PC IDS, bien qu'il ne participe pas directement à la communication, reçoit une copie complète du trafic destiné à la victime, ce qui permet une analyse passive mais en temps réel.

8.7 Simulation d'une attaque réseau

Pour tester l'efficacité de la surveillance :

- Envoyer un flux de paquets depuis le PC attaquant vers le PC victime.

- Exemples d’attaques simulées : scan de ports (ex. nmap), ping flood, injections malveillantes...

Le PC IDS intercepte ce trafic via le port miroir et peut :

- Le visualiser en temps réel
- Générer des alertes (dans le cas d’un IDS)
- Sauvegarder les paquets pour une analyse ultérieure

Attaques traitées dans le dataset

Le dataset étudié couvre environ 20 types d’attaques, parmi lesquelles on retrouve notamment :

- Les attaques de reconnaissance (Reconnaissance)
- Les attaques par déni de service (DoS)
- Les attaques par porte dérobée (Backdoor)

8.8 Attaque de Reconnaissance (Reconnaissance Attack)

Définition

Une attaque de reconnaissance constitue la phase préliminaire d’une cyberattaque. Elle consiste à collecter des informations sur la cible afin d’identifier ses vulnérabilités avant de lancer une exploitation. Elle s’apparente à une phase de surveillance dans le monde physique.

Types d’attaques de reconnaissance

A. Reconnaissance passive Cette forme de reconnaissance ne provoque pas d’interaction directe avec la cible. Elle est donc plus discrète.

- **OSINT (Open Source Intelligence)** : collecte d’informations à partir de sources publiques (réseaux sociaux, WHOIS, sites web).
Outils : theHarvester, Maltego, Google Dorks.
- **Sniffing (écoute réseau)** : capture des paquets réseau via des outils comme Wireshark ou tcpdump.
- **Ingénierie sociale (Social Engineering)** : inclut le phishing, le shoulder surfing, les appels frauduleux ou les faux profils professionnels.

B. Reconnaissance active Elle implique une interaction directe avec la cible, et est donc plus facilement détectable.

- **Scanning de réseau** : `nmap -sn`, `fping`, ARP scanning.
- **Scanning de ports** : `nmap -sS`, `nmap -sT`, `nmap -sU`.
- **Fingerprinting** :

- OS : `nmap -O`
- Services : banner grabbing avec `netcat`
- **Énumération** :
 - DNS : `dig`, `dnsrecon`
 - SNMP : `snmpwalk`
 - LDAP/AD : `ldapsearch`, `BloodHound`
- Scan de vulnérabilités : `Nessus`, `OpenVAS`, `nmap -script vuln`

8.9 Attaque par Porte Dérobée (Backdoor)

Définition

Une backdoor (porte dérobée) est un accès caché et non autorisé à un système, permettant à un attaquant de contourner les mécanismes de sécurité traditionnels, même après la résolution d'une faille initiale.

Mécanisme de fonctionnement

- Installation d'un logiciel malveillant (RAT, rootkit).
- Ouverture d'un port masqué.
- Usage de processus déguisés (exemple : `svchost.exe`).

Exemple

[language=bash] Création d'une backdoor simple avec Netcat `nc -lvp 4444 -e /bin/bash`

Types de Backdoors

A. Selon l'origine

- **Malware-based** : backdoor intégrée via des logiciels malveillants.
- **Hardware-based** : composants physiques compromis (ex. : clé USB infectée).
- **Software-based** : intentions malveillantes cachées dans le code source.

B. Selon la persistance

- **Persistante** : persiste après redémarrage (ex. : `crontab`, registre Windows).
- **Non persistante** : disparaît à l'extinction, souvent en mémoire uniquement.

Outils fréquemment utilisés

- Metasploit (Meterpreter)
- Cobalt Strike
- Netcat

Exemples d'attaques célèbres

- **SolarWinds (2020)** : backdoor introduite dans une mise à jour logicielle.
- **Equation Group** : malware avancé d'espionnage industriel.
- **Symbiote (Linux, 2023)** : rootkit furtif injecté dans les processus système.

Attaque par Déni de Service (DoS)

Définition

Une attaque DoS vise à rendre un service ou un système indisponible en surchargeant ses ressources ou en exploitant ses failles.

Objectifs

- Empêcher l'accès aux utilisateurs légitimes.
- Saturer la bande passante, le processeur ou la mémoire.
- Exploiter des bugs ou provoquer des crashes.

A. Attaques par inondation (Flooding)

- **SYN Flood** : envoi massif de requêtes SYN non finalisées. [language=Python,caption=Exemple SYN Flood avec Scapy] `from scapy.all import * send(IP(dst="192.168.1.1")/TCP(dport=80, flags="S"), loop=1)`
- **UDP Flood** : envoi massif de paquets UDP vers des ports aléatoires.
- **HTTP Flood** : surcharge par des requêtes HTTP multiples.

B. Exploitation de vulnérabilités

- **Ping of Death** : paquets ICMP anormalement volumineux.
- **Slowloris** : connexions HTTP incomplètes saturant les ressources.

C. Attaques amplifiées

- **DNS Amplification** : amplification du trafic en ciblant une victime avec des réponses massives.
- Exemple :

Attaquant → Requête DNS → Serveur DNS → Réponse amplifiée → Cible

Différences entre DoS et DDoS

^{ccc} Critère	DoS	DDoS
Origine	Machine unique	Plusieurs machines (botnet)
Échelle	Limitée	Très large (terabits/s)
Complexité	Simple	Requiert une infrastructure distribuée

Outils fréquemment utilisés

- LOIC, HOIC : flooding HTTP/UDP.
- Slowloris : attaque par saturation lente.
- hping3 : création personnalisée de paquets.

Impacts potentiels

- Interruption de services critiques (ex. : e-commerce).
- Perte financière et réputationnelle.
- Détournement d'attention pour couvrir d'autres attaques.

9 Développement de la Plateforme

Le développement de la plateforme de monitoring s'est articulé autour de la création d'une interface utilisateur intuitive et de la bonne intégration des modèles de détection afin d'offrir une expérience fluide et efficace à l'administrateur réseau.

9.1 Conception de l'interface (frontend)

- Utilisation de technologies modernes telles que **React.js** pour créer une interface dynamique et réactive.
- Affichage des résultats de détection (types d'attaques détectées).

9.2 Intégration du modèle de détection

- Mise en place d'une API backend (avec **Django REST Framework**) qui reçoit les flux réseau traités.
- Communication entre l'interface frontend et l'API via des requêtes HTTP/REST.
- Le backend exécute les modèles de *Machine Learning* et *Deep Learning* sur les données entrantes pour détecter et classifier les attaques.

9.3 Visualisation des prédictions

- Historique consultable des incidents détectés avec détails (timestamp, type d'attaque, source).

9.4 Fonctionnalité de chatbot pour l'assistance de l'administrateur

- Développement d'un chatbot spécialisé en sécurité réseau pour accompagner l'administrateur.
- Le chatbot répond aux questions liées aux alertes, définitions des types d'attaques et propose des conseils de mitigation.
- Il interroge une base de données interne et utilise des techniques de NLP (Natural Language Processing) pour comprendre et traiter les requêtes.
- Intégration directe du chatbot dans l'interface pour une interaction fluide.

10 Implémentation Technique

Cette section détaille les choix technologiques, l'architecture globale du système et les étapes du déploiement de la plateforme de ID

10.1 Technologies utilisées

- **Backend :**
 - Framework **Django** avec **Django REST Framework** pour la création d'API robustes et sécurisées.
 - Utilisation de bibliothèques Python pour le machine learning et le deep learning, notamment **scikit-learn**.
- **Frontend :**
 - Framework **React.js** pour une interface utilisateur dynamique et réactive.
- **Base de données :**
 - Base relationnelle **PostgreSQL** pour stocker les données utilisateur, les logs et les résultats d'analyse.
- **Capture réseau :**
 - Utilisation de **Scapy** et **TCPDump** pour la collecte et le traitement des paquets réseau.
- **bibliothèques :** Voici les principales bibliothèques et outils utilisés :
 - **Scikit-Learn** C'est une bibliothèque Python très populaire et complète pour le machine learning. Elle fournit des outils efficaces et faciles à utiliser pour diverses tâches telles que la classification, la régression, le clustering, la réduction de dimension et la sélection de modèles. Elle s'appuie sur des bibliothèques scientifiques fondamentales comme NumPy et SciPy pour des calculs optimisés.
 - **Pandas** Pandas est une bibliothèque Python essentielle pour la manipulation et l'analyse de données. Sa structure principale, le DataFrame, permet une gestion intuitive et performante des données tabulaires (similaires aux tableaux Excel ou aux tables de base de données). Cela est particulièrement utile pour les étapes de nettoyage, de transformation et de préparation des données avant l'application des algorithmes d'apprentissage automatique.
 - **NumPy** NumPy (Numerical Python) est la bibliothèque fondamentale pour les calculs numériques en Python. Elle offre des objets tableau multidimensionnels (arrays) très performants et une collection de fonctions pour opérer sur ces tableaux. C'est la base de nombreuses autres bibliothèques scientifiques

et elle est indispensable pour les opérations matricielles et les calculs intensifs fréquemment rencontrés en machine learning.

- **Matplotlib** Matplotlib est une bibliothèque de traçage de graphiques en Python. Elle sert à créer une grande variété de visualisations statiques, animées et interactives. Cela aide à mieux comprendre les données, à explorer leurs distributions et leurs relations, et à présenter les résultats d'analyses ou les performances des modèles de manière claire et visuelle.
- **Seaborn** Seaborn est une bibliothèque de visualisation de données Python basée sur Matplotlib. Elle est spécialisée dans la création de graphiques statistiques attrayants et informatifs. Seaborn simplifie la production de visualisations complexes et permet d'explorer plus facilement les relations entre les variables, rendant les analyses de données plus accessibles et les présentations plus esthétiques.
- **Pickle** Pickle est un module standard de Python qui permet de sérialiser et désérialiser des objets Python. Il est utilisé pour sauvegarder des objets Python (comme des modèles de machine learning entraînés, des listes ou des dictionnaires) dans des fichiers binaires, puis de les recharger ultérieurement. Cela facilite la persistance et la réutilisation des modèles sans avoir à les réentraîner, économisant ainsi du temps et des ressources.

10.2 Architecture de la solution

L'architecture suit un modèle client-serveur avec plusieurs modules :

- **Module de traitement et d'analyse** : Réceptionne les paquets, extrait les caractéristiques (*features*), puis applique les modèles d'intelligence artificielle (après prétraitement et sélection de caractéristiques) pour détecter les intrusions.
- **API REST** : Interface entre le backend et le frontend pour les requêtes classiques.
- **Frontend** : Interface utilisateur pour la visualisation et l'interaction.
- **Chatbot** : Intégré au frontend, connecté à une base de connaissances sécurisée.

10.3 Entraînement et Évaluation du Modèle

Cette section détaille le processus d'entraînement des différents modèles d'apprentissage automatique ainsi que les méthodologies et métriques utilisées pour évaluer leurs performances.

10.3.1 Entraînement du Modèle

Le processus d'entraînement a été mené en deux phases distinctes afin d'analyser l'impact de la normalisation des données sur les performances des classifieurs.

Avant Normalisation

Dans cette phase, les modèles ont été entraînés directement sur le jeu de données après l'étape d'équilibrage, mais avant toute normalisation des caractéristiques. Cela a permis d'établir une base de référence pour l'efficacité des algorithmes.

Après Normalisation

Suite à la phase initiale, une étape de normalisation des données a été appliquée. Les modèles ont ensuite été réentraînés sur ce jeu de données normalisé pour observer les améliorations potentielles en termes de convergence et de performance.

10.3.2 Évaluation des Performances du Modèle

Pour évaluer de manière exhaustive les performances des modèles entraînés, nous avons utilisé les méthodes et métriques d'évaluation suivantes, largement reconnues dans le domaine de l'apprentissage automatique et de la détection d'intrusion :

- Matrice de Confusion
- Précision (Accuracy)
- Précision (Precision)
- Rappel (Recall)
- Score F1 (F1-score)
- Taux de Faux Positifs (False Alarm Rate)

10.4 Implémentation

- **Figma** : Figma est un éditeur de graphiques vectoriels et un outil de prototypage basé sur le web. Il est principalement utilisé pour la conception d'interfaces utilisateur (UI) et offre des fonctionnalités de collaboration en temps réel exceptionnelles, permettant à plusieurs utilisateurs de travailler simultanément sur le même fichier de conception.
- **GitHub** : GitHub est une plateforme web de premier plan utilisée pour le contrôle de version, basée sur Git. Elle fournit une interface graphique intuitive, ainsi que des fonctionnalités de contrôle d'accès et de collaboration avancées, telles que le suivi des bogues, les demandes de fonctionnalités, la gestion des tâches et la documentation Wiki, facilitant le travail d'équipe sur des projets de code.
- **PuTTY** : PuTTY est un émulateur de terminal, une console série et une application de transfert de fichiers réseau open-source. Il prend en charge divers protocoles réseau, notamment SCP (Secure Copy Protocol), SSH (Secure Shell), Telnet, rlogin et les connexions via des sockets bruts, ce qui en fait un outil essentiel pour l'administration et le dépannage à distance.
- **Kaggle** : Kaggle est une communauté en ligne populaire regroupant des scientifiques des données et des praticiens de l'apprentissage automatique. Elle permet aux utilisateurs de trouver et de publier des jeux de données, d'explorer et de construire des modèles dans un environnement de science des données basé sur le web, et de collaborer avec d'autres experts du domaine.
- **Jupyter Notebook** : Jupyter Notebook est une application web open-source qui permet la création et le partage de documents interactifs. Ces documents peuvent contenir du code exécutable en direct, des équations, des visualisations et du texte narratif. Il est largement utilisé dans le nettoyage et la transformation des données, la simulation numérique, la modélisation statistique, la visualisation des données et l'apprentissage automatique pour l'exploration et la documentation.

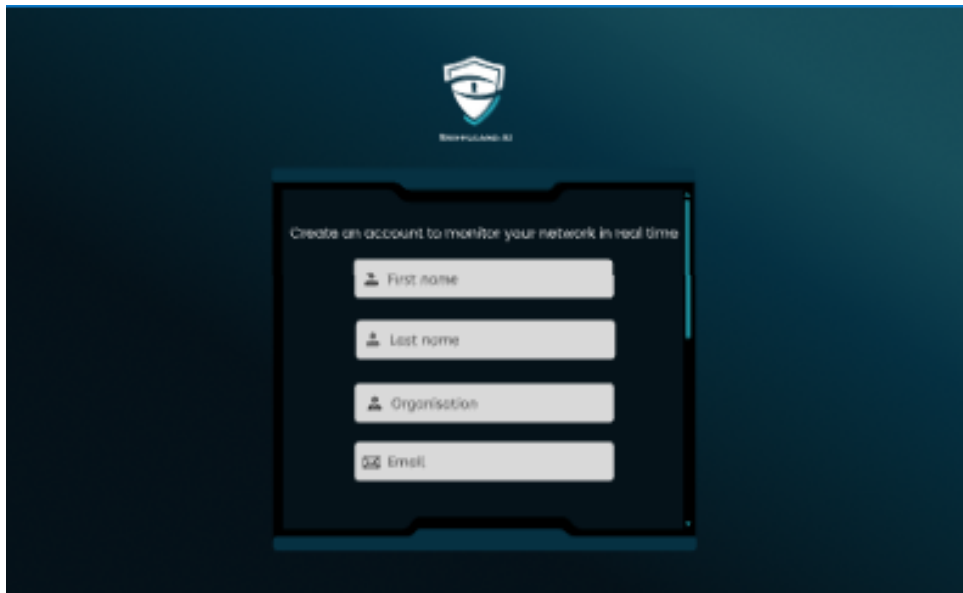
10.4.1 Détails d'Implémentation

Page d'atterrissage



FIGURE 10.1 – Page d'atterrissage

Sign Up



The image shows a sign-up form for a service called BlueFilius. At the top center is a logo consisting of a shield with a stylized 'i' inside, with the text 'BlueFILIUS.BJ' underneath it. Below the logo, the text 'Create an account to monitor your network in real time' is displayed. The form itself is a dark rectangle with a light blue border and contains four input fields, each with a small icon to its left: a person icon for 'First name', a person icon for 'Last name', a person icon for 'Organisation', and an envelope icon for 'Email'.

FIGURE 10.2 – Page de Sign Up

Page Log In

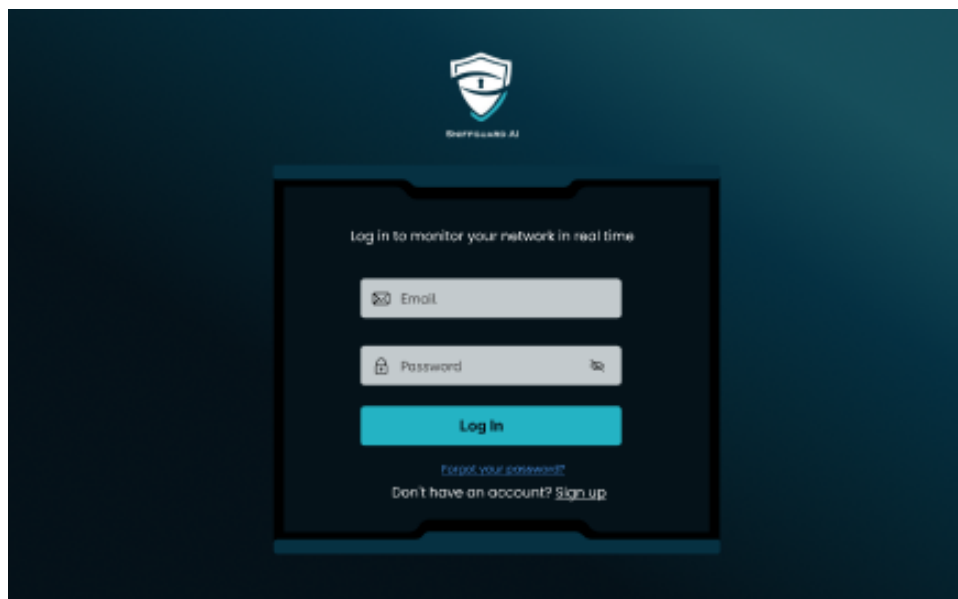


FIGURE 10.3 – Page Log In

Page de Home page



FIGURE 10.4 – Page de Home Page



CYBERSHIELD AI

Home

Scan

Live

Archive

Log out

List of Attacks Detected in Real-Time Monitoring

Attack	Detection Time	Source IP	Target IP	Protocol
DDoS	04-04-2025 09:30:05	192.168.201	192.168.201	1
Spreading	04-04-2025 09:30:05	192.168.5	192.168.5	10
DDoS	04-04-2025 09:30:05	192.168.53	192.168.53	10
Bot	04-04-2025 09:30:05	192.168.72	192.168.72	3
DDoS	04-04-2025 09:30:05	192.168.201	192.168.201	1
Spreading	04-04-2025 09:30:05	192.168.52	192.168.52	10
DDoS	04-04-2025 09:30:05	192.168.53	192.168.53	10
Bot	04-04-2025 09:30:05	192.168.72	192.168.72	3
Spreading	04-04-2025 09:30:05	192.168.52	192.168.52	10
DDoS	04-04-2025 09:30:05	192.168.53	192.168.53	10
Bot	04-04-2025 09:30:05	192.168.72	192.168.72	3

List of Attacks Detected During System Scans

Attack	Detection Time	File name	Target IP	Protocol
No Attack	04-04-2025 09:30:05	FileA	192.168.201	1
Spreading	04-04-2025 09:30:05	FileB	192.168.52	10
DDoS	04-04-2025 09:30:05	FileC	192.168.201	10
Bot	04-04-2025 09:30:05	FileD	192.168.72	3
DDoS	04-04-2025 09:30:05	FileA	192.168.201	1
No Attack	04-04-2025 09:30:05	FileA	192.168.52	10
DDoS	04-04-2025 09:30:05	FileA	192.168.201	10
Bot	04-04-2025 09:30:05	FileA	192.168.72	3
Spreading	04-04-2025 09:30:05	FileA.pcap	192.168.52	10
DDoS	04-04-2025 09:30:05	FileA.pcap	192.168.201	10
Bot	04-04-2025 09:30:05	FileA.pcap	192.168.72	3

FIGURE 10.5 – Page d'Archive

Page de Scan

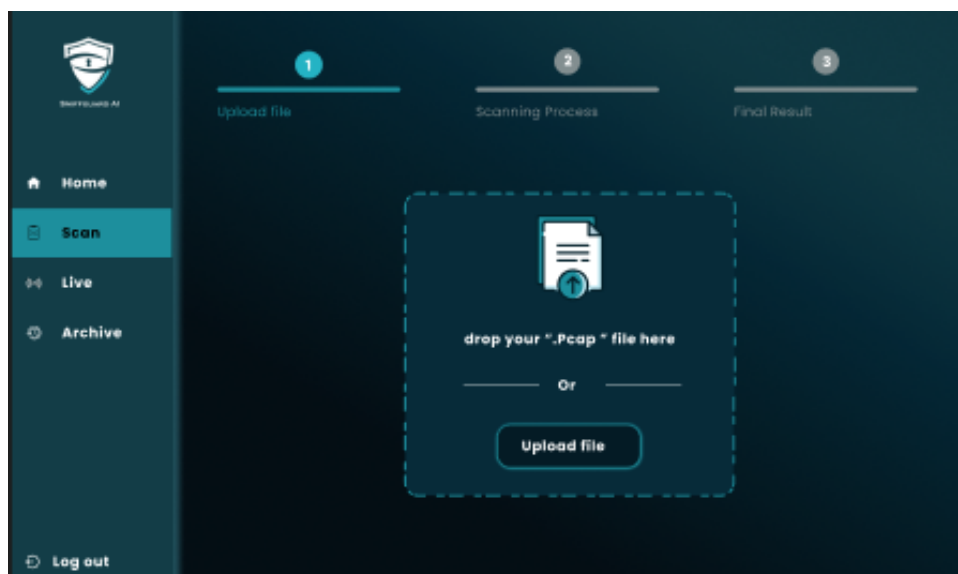


FIGURE 10.6 – Page de scan

Page de chat bot

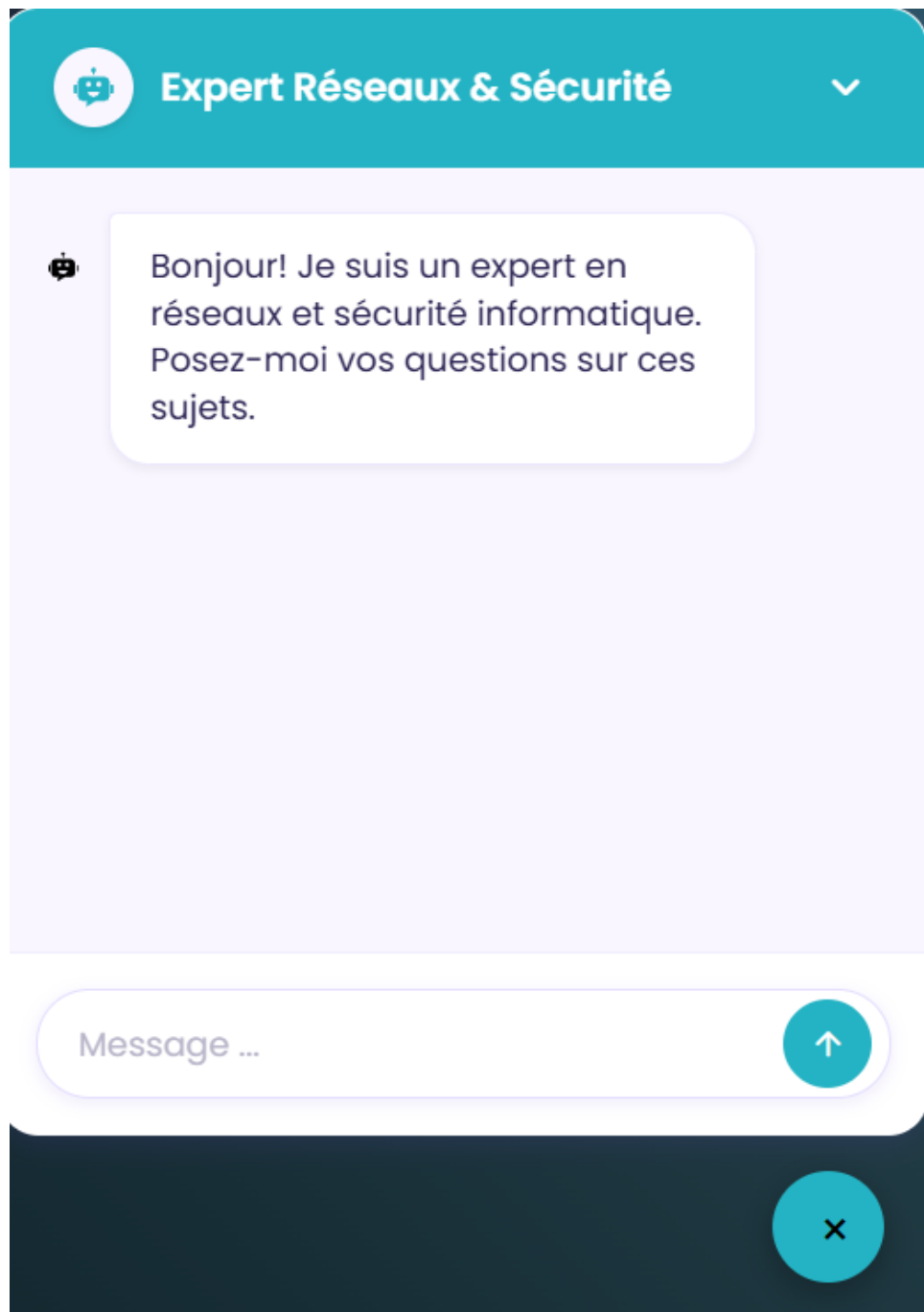


FIGURE 10.7 – Page de chat bot

11 Défis et Problèmes Rencontrés

Au cours du développement de notre système de monitoring réseau intelligent, plusieurs obstacles techniques et organisationnels ont été rencontrés. Cette section présente les principaux défis que nous avons dû relever ainsi que les solutions adoptées.

11.1 Contraintes Matérielles

- **Capacité limitée de traitement :**
 - Le dataset initial (environ 13 Go) était trop volumineux pour être traité efficacement sur nos machines.
 - *Solution* : application d'un **undersampling** pour réduire la taille, suivi d'un **oversampling** pour équilibrer les classes.
- **Temps d'entraînement des modèles :**
 - Les modèles de deep learning comme **LSTM** ou **DNN** nécessitaient des ressources importantes.
 - *Solution* : entraînement progressif sur des échantillons réduits et optimisation des paramètres pour minimiser la charge.

11.2 Problèmes de Données

- **Données déséquilibrées :**
 - La majorité des flux appartenaient à la classe « normal », rendant difficile la détection des attaques rares.
 - *Solution* : techniques de rééchantillonnage, notamment **SMOTE**.
- **Nettoyage complexe :**
 - Certaines valeurs manquantes ou incohérentes ont nécessité un traitement manuel.
 - *Solution* : création de scripts de nettoyage et standardisation.

12 Perspectives d'Amélioration

Bien que notre système de détection d'intrusions (IDS) ait démontré son efficacité dans un environnement de test, plusieurs pistes d'amélioration sont envisageables pour en faire une solution plus complète, plus réactive et mieux adaptée à des déploiements en conditions réelles.

12.1 Passage au Temps Réel et Réponse Active

- **Monitoring en temps réel** : intégrer une surveillance continue du trafic réseau avec détection instantanée des intrusions au moment de leur apparition. Cela nécessiterait une optimisation des performances des modèles et de la chaîne de traitement.
- **Réaction automatique aux attaques** : mise en place d'un système de réponse active, capable de bloquer automatiquement une adresse IP, fermer un port, ou envoyer une alerte immédiate lorsqu'une attaque est détectée.
- **Intégration avec des outils SIEM ou SOAR** : pour assurer une réponse coordonnée, la plateforme pourrait interagir avec des solutions existantes de sécurité réseau pour exécuter des contre-mesures automatiquement.

12.2 Évolution de la Plateforme

- **Tableau de bord enrichi** : ajout de graphiques interactifs, filtres dynamiques et statistiques détaillées.
- **Système d'alerte automatisé** : envoi d'alertes en temps réel via email, SMS ou webhook en cas de détection.
- **Support mobile (responsive)** : adaptation de l'interface à tous les types d'écrans pour une surveillance à distance.

13 Conclusion Générale

La sécurité des systèmes d'information représente aujourd'hui un enjeu stratégique majeur face à la multiplication des menaces et à la sophistication croissante des attaques informatiques. C'est dans ce contexte que nous avons conçu et développé un système de détection d'intrusions (IDS) s'appuyant sur une combinaison d'analyses réseau et d'approches d'intelligence artificielle avancées.

Ce projet nous a permis de mobiliser un éventail de compétences multidisciplinaires, couvrant plusieurs domaines clés :

- **Cybersécurité** : identification, classification et détection automatisée des attaques réseau à partir de données capturées sur le trafic.
- **Science des données** : traitement, analyse statistique, équilibrage et normalisation d'un dataset de grande dimension.
- **Apprentissage automatique et profond** : conception, entraînement et évaluation de modèles prédictifs de classification.
- **Réseaux informatiques** : capture et exploitation de données réseau à l'aide d'outils comme Scapy et TCPDump.
- **Développement logiciel** : construction d'une interface utilisateur claire et intuitive, intégrant un chatbot intelligent et une API pour la communication entre modules.

Les défis techniques liés à la manipulation de données volumineuses nous ont amenés à adopter des stratégies d'optimisation telles que la réduction de dimensions, l'under-sampling/oversampling et l'ajustement des hyperparamètres afin de maintenir de bonnes performances tout en limitant la complexité.

Nous avons également veillé à concevoir une plateforme interactive et modulaire, avec des interfaces dédiées à la visualisation des résultats de détection, à l'assistance via chatbot, et à l'exploitation des données réseau. Cette approche modulaire permet d'envisager facilement une extension du système vers des contextes d'utilisation plus larges, voire critiques.

En somme, ce travail constitue une base solide et évolutive pour la mise en œuvre d'un système de détection intelligent, capable d'aider les administrateurs réseau dans leur mission de sécurisation. Il illustre la synergie possible entre intelligence artificielle, cybersécurité et ingénierie logicielle au service d'une défense proactive et efficace des infrastructures informatiques.

Bibliographie

- [1] Ahmad Z., Shahid Khan A., Wai Shiang C., Abdullah J., Ahmad F.
Network intrusion detection system : A systematic study of machine learning and deep learning approaches.
Transactions on Emerging Telecommunications Technologies, 2021 ; 32 :e4150.
<https://doi.org/10.1002/ett.4150>
- [2] Kao M.-T., Sung D.-Y., Kao S.-J., Chang F.-M.
A novel two-stage deep learning structure for network flow anomaly detection.
Electronics, 11(10) :1531, 2022.
- [3] Roopak M., Tian G. Y., Chambers J.
An Intrusion Detection System Against DDoS Attacks in IoT Networks.
2020 10th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, pp. 0562–0567.
<https://doi.org/10.1109/CCWC47524.2020.9031206>
- [4] Azhari A., Muhammad A., Mohd Foozy C. F.
Machine Learning-Based Distributed Denial of Service Attack Detection on Intrusion Detection System Regarding to Feature Selection.
International Journal of Artificial Intelligence Research, 4.
<https://doi.org/10.29099/ijair.v4i1.156>
- [5] Zachos G., Essop I., Mantas G., Porfyraakis K., Ribeiro J. C., Rodriguez J.
An Anomaly-Based Intrusion Detection System for Internet of Medical Things Networks.
Electronics, 2021, 10, 2562.
<https://doi.org/10.3390/electronics10212562>
- [6] Alalhareth M., Hong S.-C.
An Improved Mutual Information Feature Selection Technique for Intrusion Detection Systems in the Internet of Medical Things.
Sensors, 2023, 23, 4971.
<https://doi.org/10.3390/s23104971>
- [7] Magdy M., Matter A., Hussin S., Hassan D., Elsaid S.
Anomaly-based intrusion detection system based on feature selection and majority voting.
Indonesian Journal of Electrical Engineering and Computer Science, 2023, 30(3) :1699–1706.
<https://doi.org/10.11591/ijeecs.v30.i3.pp1699-1706>
- [8] Sharafaldin I., Lashkari A. H., Sahib I., Ghorbani A.
Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy.

2019.

<https://doi.org/10.1109/CCST.2019.8888419>